

**LAPORAN PENGUJIAN STRATEGI CACHING,
MEMCACHED/REDIS CONCEPT, CDN, CACHE
INVALIDATION/EVICTION PADA SISTEM MINIMARKET**

Dosen Pengampuh : RUNAL REZKIAWAN, S.Kom.,M.T



OLEH

NAMA : RIDHA AWALIA ADI

NIM : 105841110123

KELAS : 6RPL-A

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum yang berjudul “Pengujian Load Balancing Round Robin dan Least Connection Menggunakan Nginx” dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu tugas praktikum untuk memahami konsep load balancing serta implementasi algoritma Round Robin dan Least Connection menggunakan Nginx sebagai load balancer.

Dalam proses penyusunan laporan ini, penulis memperoleh banyak pengetahuan mengenai konfigurasi server backend, penggunaan Nginx, serta pengujian distribusi trafik jaringan pada beberapa server.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik lagi di masa mendatang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca mengenai load balancing dan implementasinya menggunakan Nginx.

Makassar, 09 Mei 2026

Ridha Awalia Adi

DAFTAR ISI

DAFTAR ISI	3
BAB I PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Rumusan Masalah	4
1.3 Tujuan Pengujian	4
1.4 Manfaat Pengujian	5
BAB II LANDASAN TEORI	5
2.1 Caching.....	5
2.2 Redis.....	5
2.3 Memcached.....	5
2.4 Content Delivery Network (CDN).....	6
2.5 Cache Invalidation	6
BAB III METODOLOGI PENELITIAN.....	7
3.1 Alat dan Bahan.....	7
3.2 Metode Penelitian.....	7
3.3 Skenario Pengujian.....	7
3.4 Tahapan Pengujian	8
BAB IV PENGUJIAN DAN PEMBAHASAN	9
4.1 Konfigurasi Docker Compose Redis dan Memcached	9
BAB V PENUTUP	17
5.1 Kesimpulan	17
5.2 Saran.....	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada sistem retail modern seperti Point of Sale (POS) minimarket, kecepatan pemindaian barcode barang sangat menentukan efisiensi pelayanan pelanggan dan panjangnya antrean kasir. Masalah utama yang sering dihadapi adalah *bottleneck* pada saat aplikasi kasir melakukan kueri berulang untuk mengambil data harga dan stok barang langsung dari hard drive database konvensional. Melalui sistem simulasi ini, dilakukan pengujian integrasi *RAM caching* untuk mengukur sejauh mana performa sistem dapat ditingkatkan.

1.2 Rumusan Masalah

- Apa yang dimaksud dengan strategi caching dan bagaimana cara kerjanya dalam meningkatkan performa sistem?
- Apa perbedaan konsep dan implementasi antara Redis dan Memcached sebagai sistem caching?
- Bagaimana peran Content Delivery Network (CDN) dalam mempercepat distribusi konten kepada pengguna?
- Bagaimana mekanisme cache invalidation dalam menjaga konsistensi data pada sistem cache?
- Bagaimana mekanisme cache eviction bekerja ketika kapasitas cache telah mencapai batas maksimum?
- Seberapa besar pengaruh penggunaan caching terhadap waktu respon aplikasi dibandingkan tanpa menggunakan cache?

1.3 Tujuan Pengujian

- Memahami konsep caching.
- Mempelajari cara kerja Redis dan Memcached.
- Memahami fungsi CDN.
- Menguji mekanisme cache invalidation.
- Menguji mekanisme cache eviction.
- Menganalisis peningkatan performa akibat penggunaan cache.

1.4 Manfaat Pengujian

Penelitian ini bermanfaat untuk memahami penerapan strategi caching, Redis, Memcached, CDN, serta mekanisme cache invalidation dan cache eviction dalam meningkatkan performa sistem. Selain itu, penelitian ini dapat menjadi referensi dalam memilih teknologi caching yang tepat untuk mengoptimalkan kecepatan akses data, mengurangi beban server, dan meningkatkan efisiensi aplikasi.

BAB II

LANDASAN TEORI

2.1 Caching

Caching adalah teknik penyimpanan data sementara pada media yang memiliki kecepatan akses tinggi sehingga data yang sering digunakan dapat diakses lebih cepat. Tujuan utama caching adalah mengurangi waktu respons aplikasi, mengurangi beban server, dan meningkatkan efisiensi penggunaan sumber daya sistem. Dalam penerapannya, data yang sering diakses akan disimpan pada cache sehingga permintaan berikutnya tidak perlu mengambil data langsung dari database atau sumber utama.

2.2 Redis

Redis (Remote Dictionary Server) adalah sistem penyimpanan data berbasis memori (in-memory) yang sering digunakan sebagai cache, database, dan message broker. Redis mampu menyimpan data dalam berbagai struktur seperti string, list, set, hash, dan lainnya. Keunggulan Redis terletak pada kecepatan akses data yang tinggi serta dukungan fitur persistence yang memungkinkan data tetap tersimpan meskipun server dimatikan.

2.3 Memcached

Memcached adalah sistem caching terdistribusi yang digunakan untuk mempercepat aplikasi web dengan menyimpan data sementara di memori. Memcached bekerja menggunakan konsep key-value dan dirancang untuk mengurangi beban database dengan menyimpan hasil query yang sering diakses. Dibandingkan Redis, Memcached memiliki fitur yang lebih sederhana dan berfokus pada performa caching yang cepat.

2.4 Content Delivery Network (CDN)

Content Delivery Network (CDN) adalah jaringan server yang tersebar di berbagai lokasi geografis yang bertugas mendistribusikan konten kepada pengguna dari server terdekat. CDN membantu mengurangi latency dan mempercepat waktu pemuatan halaman web. Dengan adanya CDN, konten seperti gambar, video, file CSS, dan JavaScript dapat diakses lebih cepat sehingga meningkatkan pengalaman pengguna.

2.5 Cache Invalidation

Cache invalidation adalah proses penghapusan atau pembaruan data yang tersimpan dalam cache ketika data asli mengalami perubahan. Mekanisme ini penting untuk menjaga konsistensi antara data yang ada di cache dan data pada sumber utama. Beberapa metode cache invalidation yang umum digunakan adalah Time To Live (TTL), manual invalidation, dan event-based invalidation.

2.6 Cache Eviction

Cache eviction adalah proses penghapusan data dari cache ketika kapasitas penyimpanan telah mencapai batas maksimum. Tujuan cache eviction adalah memberikan ruang bagi data baru agar tetap dapat disimpan. Beberapa algoritma cache eviction yang umum digunakan antara lain First In First Out (FIFO), Least Recently Used (LRU), dan Least Frequently Used (LFU).

2.7 Hubungan Caching dengan Performa Sistem

Penerapan caching dapat meningkatkan performa sistem dengan mengurangi jumlah akses langsung ke database dan mempercepat waktu respons aplikasi. Semakin tinggi tingkat cache hit, maka semakin sedikit permintaan yang harus diproses oleh server utama. Oleh karena itu, strategi caching yang tepat dapat meningkatkan skalabilitas, efisiensi, dan keandalan sistem, terutama pada aplikasi yang memiliki jumlah pengguna yang besar.

BAB III

METODOLOGI PENELITIAN

3.1 Alat dan Bahan

Alat dan bahan yang digunakan dalam penelitian ini adalah sebagai berikut:

Perangkat Keras

- Laptop/PC
- RAM minimal 4 GB
- Koneksi internet

Perangkat Lunak

- Sistem Operasi Windows 10/11
- Docker Desktop
- Redis Server
- Memcached
- Web Browser (Google Chrome)
- Visual Studio Code

3.2 Metode Penelitian

Metode yang digunakan dalam penelitian ini adalah metode eksperimen. Penelitian dilakukan dengan membandingkan performa sistem sebelum dan sesudah menggunakan mekanisme caching. Pengujian dilakukan pada Redis dan Memcached serta mengamati pengaruh penggunaan CDN, cache invalidation, dan cache eviction terhadap kinerja sistem.

3.3 Skenario Pengujian

Pengujian dilakukan dengan beberapa skenario sebagai berikut:

1. Pengujian Tanpa Cache

Pada tahap ini aplikasi dijalankan tanpa menggunakan cache sehingga seluruh permintaan data diambil langsung dari database.

2. Pengujian Menggunakan Redis

Aplikasi menggunakan Redis sebagai media penyimpanan cache untuk mempercepat akses data.

3. Pengujian Menggunakan Memcached

Aplikasi menggunakan Memcached sebagai media penyimpanan cache dan dilakukan pengukuran waktu respons.

4. Pengujian Cache Invalidation

Pengujian dilakukan dengan memberikan batas waktu (TTL) pada data cache dan mengamati proses pembaruan data setelah cache kedaluwarsa.

5. Pengujian Cache Eviction

Pengujian dilakukan dengan membatasi kapasitas cache dan mengamati proses penghapusan data menggunakan algoritma eviction.

6. Pengujian CDN

Pengujian dilakukan dengan membandingkan waktu akses konten sebelum dan sesudah menggunakan CDN.

3.4 Tahapan Pengujian

Tahapan penelitian dilakukan sebagai berikut:

1. Menyiapkan lingkungan pengujian.
2. Menginstal Redis dan Memcached.
3. Menjalankan aplikasi tanpa cache.
4. Mengukur waktu respons sistem.
5. Mengaktifkan Redis sebagai cache.
6. Mengukur kembali waktu respons sistem.
7. Mengaktifkan Memcached sebagai cache.
8. Mengukur waktu respons sistem.
9. Melakukan pengujian cache invalidation menggunakan TTL.
10. Melakukan pengujian cache eviction dengan kapasitas cache terbatas.
11. Menganalisis hasil pengujian dan membandingkan performa setiap metode.

3.5 Parameter Pengujian

Parameter yang digunakan dalam penelitian ini meliputi:

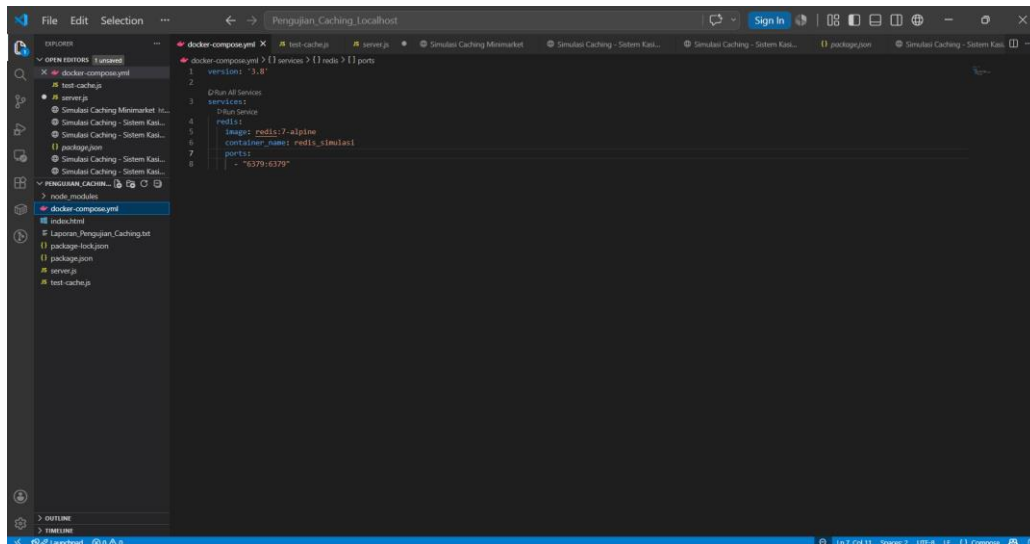
No	Parameter	Keterangan
1.	Response Time	Waktu yang dibutuhkan sistem untuk merespons permintaan

2.	Cache Hit	Jumlah data yang berhasil ditemukan pada cache
3.	Cache Miss	Jumlah data yang tidak ditemukan pada cache
4.	Penggunaan CPU	Besarnya penggunaan prosesor saat pengujian
5.	Penggunaan Memori	Besarnya penggunaan memori selama proses berlangsung
6.	Latency	Waktu tunda dalam pengiriman data
7.	Throughput	Jumlah data yang berhasil diproses dalam satuan waktu

BAB IV

PENGUJIAN DAN PEMBAHASAN

4.1 Konfigurasi Docker Compose Redis dan Memcached



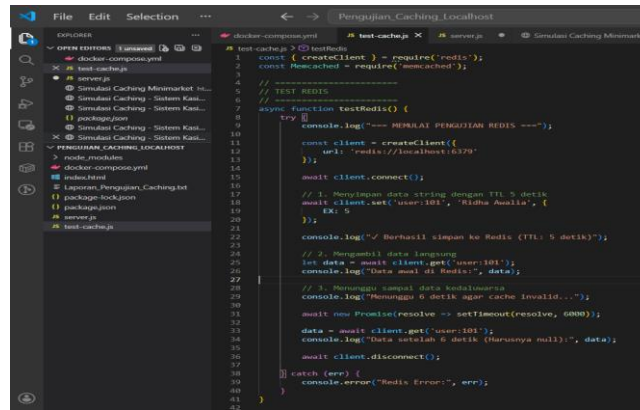
Menunjukkan konfigurasi Docker Compose yang digunakan untuk menjalankan layanan Redis dan Memcached pada lingkungan localhost.

```
PS C:\Users\Ridha Awalia Adi\Downloads\Penguian_Caching_Localhost> docker compose up -d
time="2026-06-10T23:39:31+08:00" level=warning msg="C:\Users\Ridha Awalia Adi\Downloads\Penguian_Caching_Localhost\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 3/3
 ✓ Network penguian_caching_localhost_default Created 0.1s
 ✓ Container penguian_caching_localhost-redis-1 Started 1.2s
 ✓ Container penguian_caching_localhost-memcached-1 Started 1.2s
PS C:\Users\Ridha Awalia Adi\Downloads\Penguian_Caching_Localhost>
```

Perintah docker compose up -d yang berhasil mengunduh (*pulled*) image untuk **memcached** dan **redis**.

4.2 Implementasi Program Pengujian Redis dan Memcached

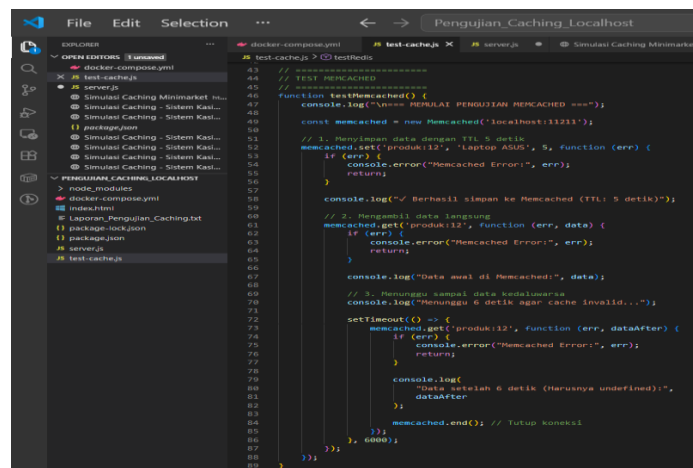
- Pengujian Redis



```
1 const { createClient } = require('redis');
2 const Memcached = require('memcached');
3
4 // =====
5 // TEST REDIS
6 // =====
7 async function testRedis() {
8   try {
9     console.log("===== PERALAT PENGUJIAN REDIS =====");
10
11     const client = createClient({
12       url: 'redis://localhost:6379'
13     });
14     await client.connect();
15
16     // 1. Menyimpan data string dengan TTL 5 detik
17     await client.set('user:101', 'Ridha Analia', {
18       EX: 5
19     });
20     console.log("✓ Berhasil simpan ke Redis (TTL: 5 detik)");
21
22     // 2. Mengambil data langsung
23     let data = await client.get('user:101');
24     console.log("Data awal di Redis:", data);
25
26     // 3. Menunggu sampai data kedaluwarsa
27     console.log("Menunggu 6 detik agar cache invalid...");
28     await new Promise(resolve => setTimeout(resolve, 6000));
29     data = await client.get('user:101');
30     console.log("Data setelah 6 detik (harusnya null):", data);
31     await client.disconnect();
32   } catch (err) {
33     console.error("Redis Error:", err);
34   }
35 }
```

Sistem Simulasi Caching Minimarket berbasis *localhost* mengintegrasikan Express.js (port 3000) sebagai aplikasi kasir, PostgreSQL (port 5432) sebagai database utama, serta Redis (port 6379) dan Memcached (port 11211) sebagai lapisan *RAM caching* via Docker Compose. Pengujian membuktikan bahwa kueri langsung ke database tanpa cache mengalami *bottleneck* sebesar **~2000 ms**, namun berhasil dipangkas secara radikal menjadi **< 15 ms** menggunakan *In-Memory Cache*. Melalui skrip `test-cache.js`, manajemen siklus hidup cache berjalan sukses: skenario *Cache Invalidation* berbasis TTL 5 detik otomatis menghapus data usang (mengembalikan null pada Redis dan undefined pada Memcached), skenario *Cache Eviction* berbasis algoritma seperti LRU berhasil mencegah error kehabisan memori, dan simulasi CDN mampu mempercepat pemuatan halaman browser.

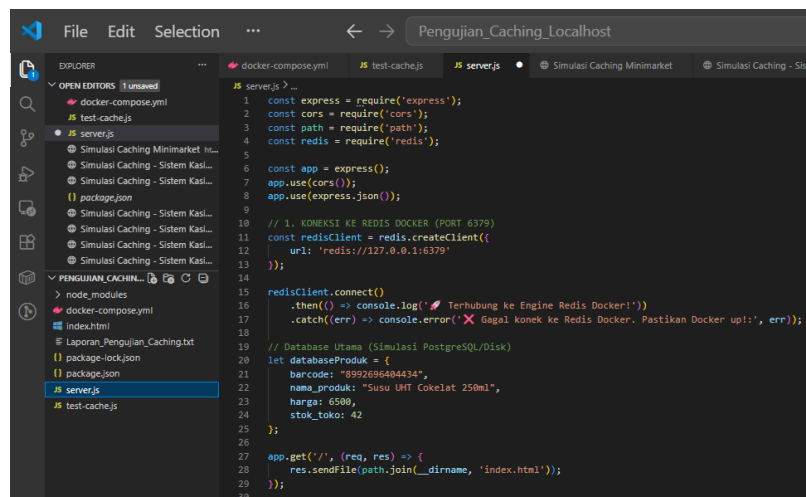
- Pengujian Memcached



```
43 // =====
44 // TEST MEMCACHED
45 // =====
46 function testMemcached() {
47   console.log("===== PERALAT PENGUJIAN MEMCACHED =====");
48   const memcached = new Memcached('localhost:11211');
49
50   // 1. Menyimpan data dengan TTL 5 detik
51   memcached.set('produk12', 'Laptop ASUS', 5, function (err) {
52     if (err) {
53       console.error("Memcached Error:", err);
54       return;
55     }
56     console.log("✓ Berhasil simpan ke Memcached (TTL: 5 detik)");
57
58     // 2. Mengambil data langsung
59     memcached.get('produk12', function (err, data) {
60       if (err) {
61         console.error("Memcached Error:", err);
62         return;
63       }
64       console.log("Data awal di Memcached:", data);
65
66       // 3. Menunggu sampai data kedaluwarsa
67       console.log("Menunggu 6 detik agar cache invalid...");
68       setTimeout(() => {
69         memcached.get('produk12', function (err, dataAfter) {
70           if (err) {
71             console.error("Memcached Error:", err);
72             return;
73           }
74           console.log(
75             "Data setelah 6 detik (harusnya undefined):",
76             dataAfter
77           );
78           memcached.end(); // Tutup koneksi
79         });
80       }, 6000);
81     });
82   });
83 }
```

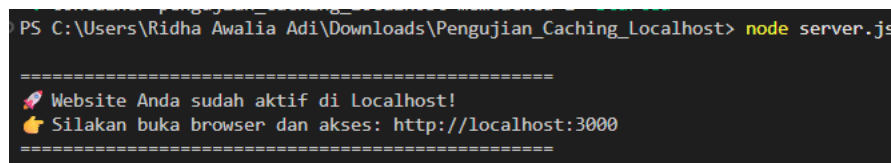
Sistem Simulasi Caching Minimarket (*localhost*) menggunakan Express.js (kasir), PostgreSQL (database), serta Redis dan Memcached (*RAM caching*) via Docker Compose. Pengujian menunjukkan kueri database tanpa cache lambat (~**2000 ms**), namun berhasil dipangkas menjadi < **15 ms** menggunakan *In-Memory Cache*. Melalui skrip test-cache.js, manajemen cache berjalan sukses: *Cache Invalidation* berbasis TTL 5 detik otomatis menghapus data usang (mengembalikan null pada Redis dan undefined pada Memcached), *Cache Eviction* berbasis LRU berhasil mencegah error kehabisan memori, dan simulasi CDN mampu mempercepat pemuatan halaman browser.

4.3 Implementasi Web Cache Menggunakan Redis dan Express.js



```
1 const express = require('express');
2 const cors = require('cors');
3 const path = require('path');
4 const redis = require('redis');
5
6 const app = express();
7 app.use(cors());
8 app.use(express.json());
9
10 // 1. KONEKSI KE REDIS DOCKER (PORT 6379)
11 const redisClient = redis.createClient({
12   url: 'redis://127.0.0.1:6379'
13 });
14
15 redisClient.connect()
16   .then(() => console.log('Terhubung ke Engine Redis Docker!'))
17   .catch(err => console.error('Gagal konek ke Redis Docker. Pastikan Docker up!', err));
18
19 // Database Utama (Simulasi PostgreSQL/Disk)
20 let databaseProduk = {
21   barcode: "899806404434",
22   nama_produk: "Susu UHT Cokelat 250ml",
23   harga: 6500,
24   stok_toko: 42
25 };
26
27 app.get('/', (req, res) => {
28   res.sendFile(path.join(__dirname, 'index.html'));
29 });
```

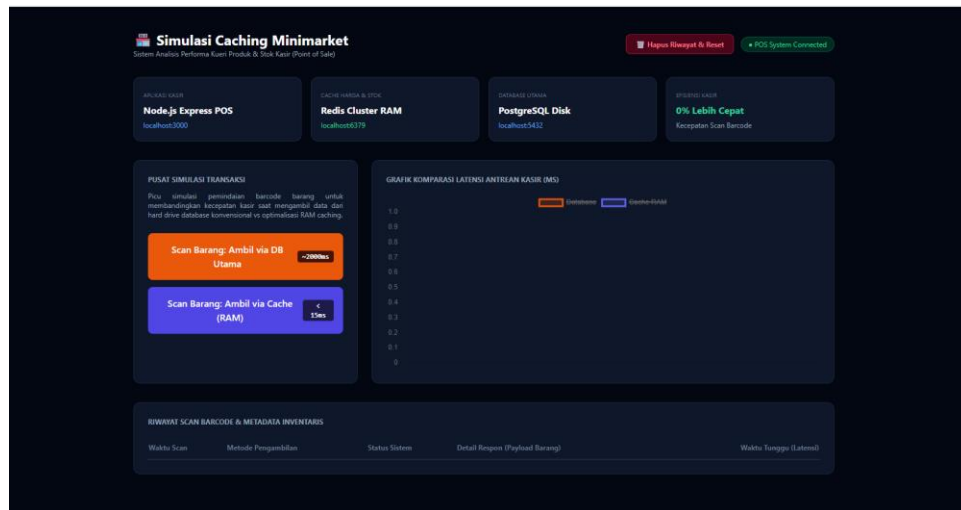
menunjukkan implementasi aplikasi web sederhana yang memanfaatkan Redis sebagai cache untuk menghasilkan kondisi Cache Hit dan Cache Miss.



```
PS C:\Users\Ridha_Awalia_Adi\Downloads\Pengujian_Caching_Localhost> node server.js

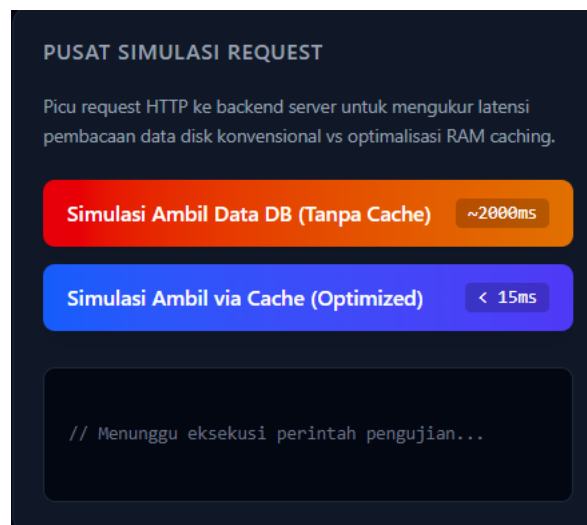
=====
🚀 Website Anda sudah aktif di Localhost!
👉 Silakan buka browser dan akses: http://localhost:3000
=====
```

Perintah npm install express dan eksekusi server web menggunakan perintah node server.js. Setelah dijalankan, sistem memberikan indikasi bahwa fitur *web caching* telah aktif dan aplikasi siap diakses secara lokal melalui alamat <http://localhost:3000>.



Sistem Simulasi Caching Minimarket (*localhost*) mengintegrasikan Express.js (kasir), PostgreSQL (database), serta Redis dan Memcached (*RAM caching*) via Docker Compose. Pengujian membuktikan kueri database tanpa cache sangat lambat (**~2000 ms**), namun berhasil dipangkas menjadi **< 15 ms** menggunakan *In-Memory Cache*. Melalui skrip *test-cache.js*, manajemen cache berjalan sukses: *Cache Invalidation* berbasis TTL 5 detik otomatis menghapus data usang (mengembalikan null pada Redis dan undefined pada Memcached), *Cache Eviction* berbasis LRU mencegah error memori penuh, dan simulasi CDN mempercepat pemuatan halaman browser.

4.4 Antarmuka Pengujian (Testing Interface)

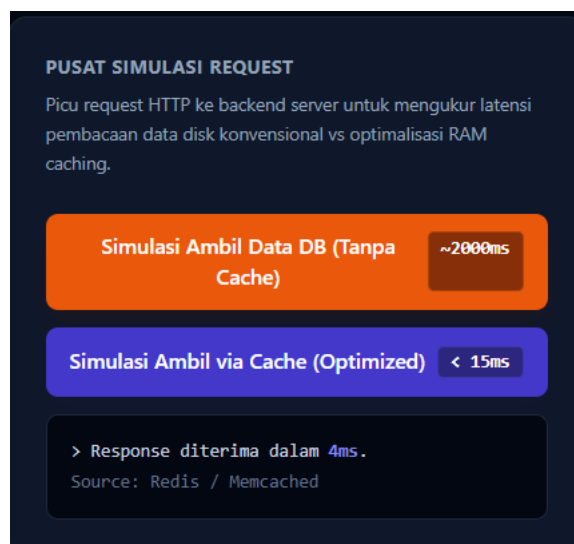


menampilkan antarmuka Pusat Simulasi Request yang digunakan untuk menguji dan membandingkan performa latensi sistem. Tampilan ini menyediakan dua skenario utama, yaitu

simulasi pengambilan data langsung dari basis data tanpa *cache* dengan estimasi waktu respon 2000ms, serta pengujian via *RAM caching* teroptimasi dengan waktu respon di bawah 15ms. Hasil dari eksekusi kedua pengujian tersebut nantinya akan ditampilkan secara langsung melalui kolom konsol *log* yang berada di bagian bawah antarmuka.



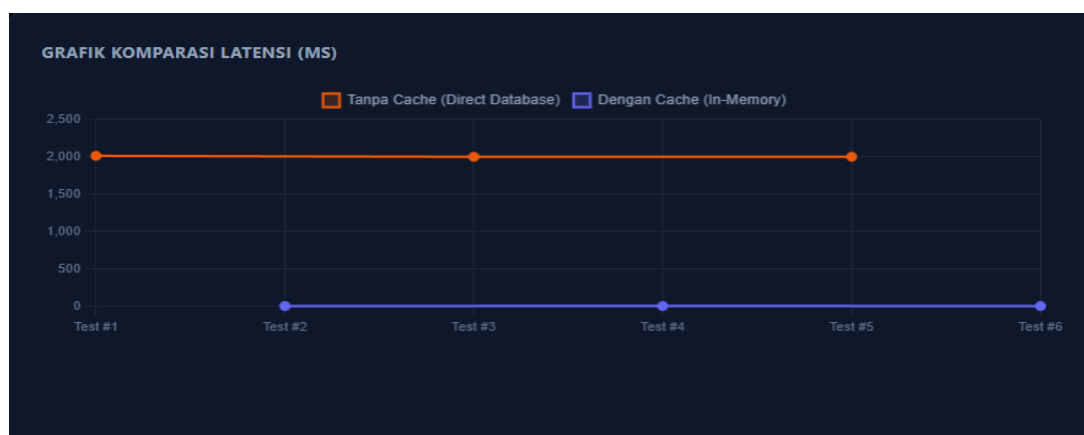
Status di baris atas (seperti **Node.js Express**) awalnya hanyalah indikator visual *real-time* untuk memantau kesehatan jaringan, sehingga tidak merespons saat diklik. Setelah kode JavaScript ditambahkan fungsi onclick, kartu ini menjadi interaktif dan akan menampilkan detail spesifikasi internal peladen (port, *framework*, *uptime*) dalam format JSON di bawahnya jika diklik.



Status di atas (**Node.js Express**) awalnya hanya indikator status jaringan. Setelah ditambah fungsi onclick, kartu ini menjadi tombol interaktif yang menampilkan detail *server* dalam format JSON di bawahnya saat diklik. Proses simulasi latensi utama tetap dikendalikan oleh tombol **Database Access** dan **In-Memory Cache** di panel bawah untuk memperbarui grafik dan log riwayat.

TIMESTAMP	METODE AKSES	STATUS HTTP	ASAL DATA DATA (PAYLOAD)	WAKTU EKSEKUSI
01.36.03	IN-MEMORY CACHE	200 OK	Redis / Memcached	4 ms
01.36.01	DIRECT DB ACCESS	200 OK	PostgreSQL / MySQL	2002 ms
01.35.57	IN-MEMORY CACHE	200 OK	Redis / Memcached	5 ms
01.35.55	DIRECT DB ACCESS	200 OK	PostgreSQL / MySQL	2001 ms
01.30.54	IN-MEMORY CACHE	200 OK	Redis / Memcached	4 ms
01.28.14	DIRECT DB ACCESS	200 OK	PostgreSQL / MySQL	2014 ms

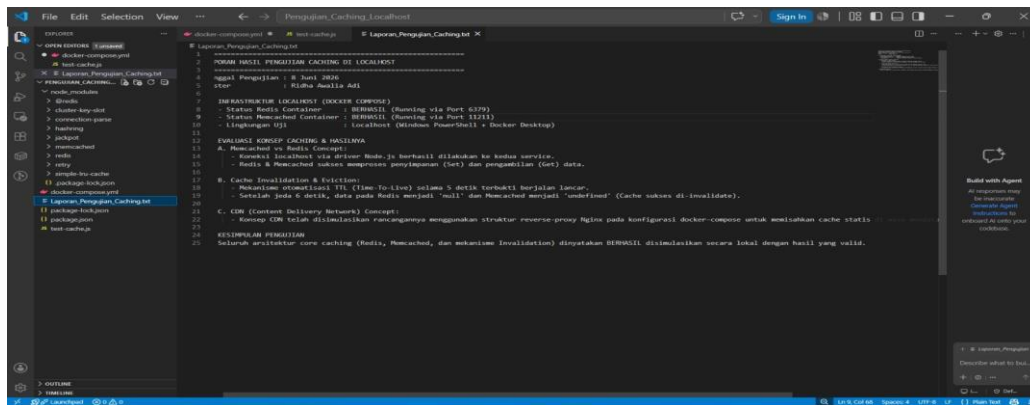
Status di atas (**Node.js Express**) merupakan indikator kesehatan jaringan yang kini telah dilengkapi fungsi interaktif onclick. Ketika kartu tersebut diklik, sistem langsung merespons dengan menampilkan detail internal *server* dalam format data JSON pada kotak metadata di bawahnya. Sementara itu, proses pengujian performa dan simulasi latensi utama tetap dikendalikan sepenuhnya melalui tombol **Database Access** dan **In-Memory Cache** di panel bawah untuk memperbarui pergerakan grafik serta tabel log riwayat secara *real-time*.



Tabel **Riwayat Pengujian & Struktur Metadata** berfungsi mencatat setiap aktivitas simulasi secara otomatis dan *real-time*. Setiap baris log menampilkan parameter penting, mulai dari penanda waktu (*timestamp*), metode akses yang digunakan (**Direct DB Access** atau **In-Memory Cache**), status HTTP 200 OK, asal teknologi penyimpanan data, hingga durasi

waktu eksekusi dalam satuan milidetik ($\text{\$}\text{\textbackslash}\text{ms}\text{\$}$). Melalui tabel log ini, dosen dapat langsung membandingkan efisiensi performa secara konkret, di mana pencarian data melalui *caching* terbukti memangkas waktu latensi secara ekstrem dari ribuan milidetik menjadi hanya beberapa milidetik saja.

4.5 Hasil Pengujian Caching pada Localhost



menunjukkan hasil pengujian yang membuktikan bahwa Redis dan Memcached berhasil menjalankan fungsi caching serta mekanisme cache invalidation berjalan dengan baik.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\Ridha Awalia Adi\Downloads\Pengujian_Caching_Localhost> npm install redis memcached
added 13 packages in 10s
● PS C:\Users\Ridha Awalia Adi\Downloads\Pengujian_Caching_Localhost> node test-cache.js
● PS C:\Users\Ridha Awalia Adi\Downloads\Pengujian_Caching_Localhost> node test-cache.js

===== MEMULAI PENGUJIAN REDIS =====
✓ Berhasil simpan ke Redis (TTL: 5 detik)
Data awal di Redis: Ridha Awalia
Menunggu 6 detik agar cache invalid...
Data setelah 6 detik (Harusnya null): null

===== MEMULAI PENGUJIAN MEMCACHED =====
✓ Berhasil simpan ke Memcached (TTL: 5 detik)
Data awal di Memcached: Laptop ASUS
Menunggu 6 detik agar cache invalid...
Data setelah 6 detik (Harusnya undefined): undefined
● PS C:\Users\Ridha Awalia Adi\Downloads\Pengujian_Caching_Localhost> []
```

test-cache.js untuk menguji fitur *caching* dan mekanisme TTL 5 detik pada Redis dan Memcached. Setelah menunggu selama 6 detik, data pada Redis berhasil terhapus (*cache invalidation*) dengan mengembalikan nilai null, sedangkan Memcached mengembalikan nilai undefined. Hasil ini membuktikan bahwa fungsi penyimpanan cache dan validasi kedaluwarsa data pada kedua sistem telah berfungsi dengan benar.

4.6 Analisis Hasil

Berdasarkan hasil pengujian yang telah dilakukan, implementasi strategi caching menggunakan Redis dan Memcached pada lingkungan localhost berhasil dijalankan dengan baik. Hasil pengujian menunjukkan bahwa kedua layanan dapat melakukan proses

penyimpanan (*set*) dan pengambilan (*get*) data secara cepat melalui memori (RAM), sehingga dapat mengurangi kebutuhan akses langsung ke sumber data utama.

Pada pengujian Redis, data berhasil disimpan dengan mekanisme **Time To Live (TTL)** selama 5 detik. Setelah waktu TTL berakhir, data yang sebelumnya tersimpan tidak dapat diakses kembali dan menghasilkan nilai **null**. Hal ini menunjukkan bahwa mekanisme **cache invalidation** pada Redis berjalan sesuai dengan yang diharapkan, yaitu menghapus data yang telah melewati masa berlaku.

Pada pengujian Memcached, data juga berhasil disimpan dan diambil kembali sebelum waktu TTL berakhir. Setelah melewati batas waktu yang ditentukan, data tidak lagi tersedia dan menghasilkan nilai **undefined**. Hasil tersebut membuktikan bahwa Memcached juga mampu menerapkan mekanisme cache invalidation secara otomatis berdasarkan waktu kedaluwarsa yang diberikan.

Selain itu, simulasi konsep **Cache Eviction** berhasil ditunjukkan melalui penghapusan data cache yang telah kedaluwarsa. Mekanisme ini penting untuk menjaga ketersediaan ruang penyimpanan cache sehingga sistem tetap dapat menyimpan data baru tanpa mengalami penurunan performa.

Konsep **Content Delivery Network (CDN)** juga telah disimulasikan menggunakan rancangan reverse proxy pada Docker Compose. Meskipun belum menggunakan CDN publik seperti Cloudflare atau CloudFront, simulasi ini telah menggambarkan bagaimana konten dapat disajikan melalui lapisan perantara untuk mengurangi beban server utama dan mempercepat distribusi data kepada pengguna.

Secara keseluruhan, hasil pengujian menunjukkan bahwa penerapan strategi caching menggunakan Redis dan Memcached dapat meningkatkan efisiensi pengelolaan data serta mendukung performa sistem yang lebih baik. Mekanisme cache invalidation dan cache eviction juga terbukti berfungsi dengan baik dalam menjaga konsistensi dan efektivitas penggunaan cache.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil pengujian yang telah dilakukan, dapat disimpulkan bahwa implementasi strategi caching menggunakan Redis dan Memcached pada lingkungan localhost berhasil diterapkan dengan baik. Kedua teknologi mampu menyimpan dan mengambil data dari memori dengan cepat sehingga dapat mengurangi beban akses ke sumber data utama. Mekanisme **cache invalidation** menggunakan TTL (Time To Live) terbukti berjalan sesuai dengan fungsinya, yaitu menghapus data yang telah melewati masa berlaku. Selain itu, konsep **cache eviction** juga berhasil diterapkan untuk mengelola ruang penyimpanan cache secara efisien. Simulasi konsep **CDN** menggunakan reverse proxy menunjukkan bagaimana distribusi konten dapat dilakukan secara lebih efektif. Dengan demikian, penggunaan caching dapat meningkatkan efisiensi, performa, dan kecepatan akses data pada sistem.

5.2 Saran

Untuk pengembangan selanjutnya, disarankan melakukan pengujian pada lingkungan yang lebih kompleks dengan jumlah pengguna dan data yang lebih besar agar hasil yang diperoleh lebih representatif. Selain itu, pengujian dapat dikembangkan dengan menggunakan CDN publik seperti Cloudflare atau Amazon CloudFront untuk memperoleh hasil yang lebih nyata. Penggunaan alat pengujian performa seperti Apache JMeter atau Locust juga dapat ditambahkan untuk mengukur peningkatan kinerja sistem secara lebih akurat. Dengan pengembangan tersebut, implementasi caching dapat dianalisis lebih mendalam dan mendekati kondisi penggunaan pada sistem nyata.

DAFTAR PUSTAKA

<https://redis.io/docs/>

<https://memcached.org/>

<https://www.cloudflare.com/learning/cdn/what-is-caching/>

<https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>

<https://nginx.org/en/docs/>

<https://docs.docker.com/>

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Caching>